Análise de Dados Reais

Segurança Pública do Estado do Rio de Janeiro

Prof. Cassio Ribeiro

Contextualização da Base de Dados

Base ISP

- **ISP-RJ**
  - Instituto de Segurança Pública do Estado do Rio de Janeiro
  - Órgão responsável por produzir e divulgar estatísticas criminais
- **CISP**
  - Circunscrição Integrada de Segurança Pública
  - Unidade territorial de cada delegacia de polícia
- **Base utilizada**
  - Evolução Mensal de Ocorrências por CISP
  - Fonte: https://www.ispdados.rj.gov.br
- **Estrutura da base**
  - Vamos descobrir juntos com .shape

Carregando o CSV com Pandas

pandas

```
import pandas as pd
import numpy as np

url = 'https://www.ispdados.rj.gov.br/Arquivos/BaseDPEvolucaoMensalCisp.csv'

df = pd.read_csv(url, encoding='latin-1', sep=';')
```

- **Por que encoding='latin-1'?**
  - O arquivo contém caracteres especiais do português (ã, ç, é...)
  - Sem esse parâmetro o pandas não consegue ler o arquivo corretamente

- **Por que sep=';'?**
  - Este CSV usa ponto e vírgula como delimitador no lugar da vírgula
  - O pandas precisa saber qual caractere separa as colunas

Explorando o DataFrame

pandas

`.shape`

Dimensões
(linhas × colunas)

`.dtypes`

Tipo de dado
de cada coluna

`.head()`

Exibe as
primeiras 5 linhas

`.info()`

Tipos, nulos
e memória

`.describe()`

Estatísticas iniciais:
média, min, max…

`.tail()`

Exibe as
últimas 5 linhas

Variáveis Qualitativas vs. Quantitativas

conceito

Qualitativas

Classificam ou identificam categorias e grupos

- `munic`
- `cisp`
- `aisp`
- `mes_ano`

Quantitativas

Expressam quantidades mensuráveis numericamente

- `furto_celular`
- `roubo_veiculos`
- `furto_veiculo`

groupby: Furto de Celular por CISP

pandas

Conceito

Agrupar linhas do DataFrame por uma variável qualitativa e aplicar uma operação sobre os valores numéricos de cada grupo.

```
# Agrupando por CISP e somando os furtos de celular
df_cisp = df.groupby('cisp')['furto_celular'].sum().reset_index()

# Ordenando do maior para o menor
df_cisp = df_cisp.sort_values(by='furto_celular', ascending=False)
```

- **reset_index()**
  - Transforma o resultado em um DataFrame com colunas nomeadas
- **sort_values(by='furto_celular')**
  - Com reset_index() o nome da coluna precisa ser informado no parâmetro by

Filtragem com .loc e .query()

pandas

Contexto: analisar apenas a cidade do Rio de Janeiro (capital)

```
# Usando .loc
df_capital = df.loc[df['munic'] == 'Rio de Janeiro']

# Usando .query() — sintaxe alternativa mais legível
df_capital = df.query("munic == 'Rio de Janeiro'")

# Repetindo o groupby apenas para a capital
df_cisp_capital = df_capital.groupby('cisp')['furto_celular'].sum().reset_index()
df_cisp_capital = df_cisp_capital.sort_values(by='furto_celular', ascending=False)
```

- **Comparando os resultados**
  - O ranking de CISPs muda quando olhamos só para a capital?
  - Quais delegacias aparecem no topo nos dois cenários?

O que é um Array?

numpy

Array é uma estrutura de dados do NumPy otimizada para operações matemáticas e estatísticas sobre sequências de valores numéricos.

Criando o array a partir da coluna do DataFrame df_cisp_capital:

```
# df_cisp_capital já foi criado no passo anterior (groupby + reset_index)
array_furto = np.array(df_cisp_capital['furto_celular'])

print(array_furto)
```

- **Por que criar um array?**
  - As funções estatísticas do NumPy (mean, median, percentile) operam sobre arrays
  - Performance superior para cálculos matemáticos em grandes volumes de dados

Média e Mediana

numpy

```
media  = np.mean(array_furto)
mediana = np.median(array_furto)

print('Média:  ', media)
print('Mediana:', mediana)
```

np.mean()

Soma de todos os valores
dividida pela quantidade de dados.
Sensível a valores extremos.

np.median()

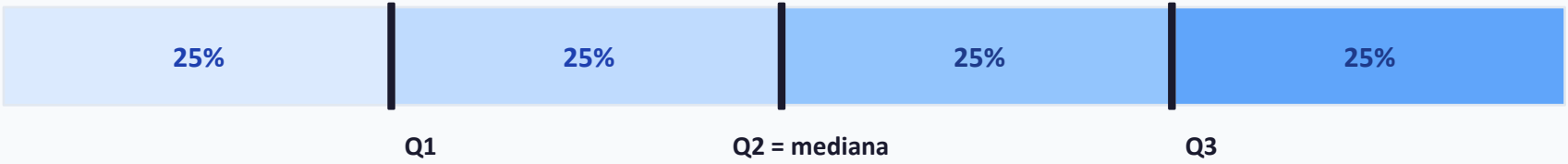
Valor central que divide os dados
em duas metades iguais.
Q2 da distribuição.

Quartis: Linhas de Corte da Distribuição

numpy

```
q1 = np.percentile(array_furto, 25)
q2 = np.percentile(array_furto, 50)
q3 = np.percentile(array_furto, 75)

print('Q1:', q1)
print('Q2:', q2)
print('Q3:', q3)
```



- **Importante:**
  - Esses valores são linhas de corte — não necessariamente existem como valores reais na variável
  - Q2 é igual à mediana: divide os dados exatamente ao meio

Aplicando os Quartis na Análise

pandas + numpy

CISPs nos 25% com mais furtos de celular — acima de Q3:

```
# Usando .loc
df_top25 = df_cisp_capital.loc[df_cisp_capital['furto_celular'] > q3]

# Usando .query()
df_top25 = df_cisp_capital.query('furto_celular > @q3')

print(df_top25)
```

- **Interpretação**
  - Delegacias acima de Q3 estão entre os 25% com maior volume de furtos de celular
  - Esse filtro identifica os pontos mais críticos da distribuição

- **Pergunta analítica**
  - Quais CISPs estão nos 25% com mais furtos de celular?

Exercício 1

prática

Análise de roubo de veículos por CISP

- Utilizando a variável roubo_veiculos:
  - 1. Crie o groupby por CISP com soma e reset_index()
  - 2. Ordene do maior para o menor
  - 3. Crie o array numpy a partir da coluna roubo_veic
  - 4. Calcule e exiba a média e a mediana
  - 5. Calcule Q1, Q2 e Q3 em variáveis separadas
  - 6. Filtre as CISPs que estão acima de Q3
- Quais CISPs estão nos 25% com mais roubos de veículo?
- Quais CISPs estão nos 25% com menos roubos de veículo?

Exercício 2

prática

Análise de furto de veículos na Baixada Fluminense

- Filtre os municípios da Baixada Fluminense:

- Com os dados filtrados:
  - 1. Crie o groupby por CISP com soma e reset_index() usando furto_veiculo
  - 2. Crie o array numpy a partir da coluna furto_veiculo
  - 3. Calcule Q1, Q2 e Q3 em variáveis separadas
  - 4. Filtre as CISPs acima de Q3

- Quais CISPs da Baixada estão nos 25% com mais furtos de veículo?
- Quais CISPs da Baixada estão nos 25% com menos furtos de veículo?